

Trabalho 4 de Estruturas de Dados

Sistema de Autocomplete de Jogos com Árvore Trie

Gabriel Bittencourt Dias

Lucas Gabriel de Oliveira

1 Introdução

Este relatório descreve um sistema de *autocomplete* em C++ baseado em uma árvore Trie, projetado para buscar títulos de jogos a partir de prefixos. O algoritmo de busca ignora espaços e a distinção entre maiúsculas e minúsculas, processando apenas caracteres alfanuméricos. O sistema retorna até k resultados, ordenados por popularidade decrescente e desempatados por ordem alfabética.

2 Estruturas de Dados Utilizadas

2.1 Classe Game

A entidade base é a classe **Game**, que encapsula os atributos de cada jogo (`title`, `shortDescription` e `popularity`). A classe fornece métodos de acesso para proteger a integridade dos dados durante a execução.

2.2 A Árvore Trie e seus Nós

A estrutura de armazenamento foi implementada através da classe **Trie**, composta por nós do tipo **TrieNode**. Cada nó possui:

- Um vetor estático de 36 ponteiros para os filhos, englobando as 26 letras (a-z) e os 10 dígitos numéricos (0-9).
- Um booleano `isEndOfTitle`, demarcando se o caminho até o nó forma uma palavra válida no catálogo.
- Um ponteiro para o objeto **Game** original, permitindo o resgate imediato de suas informações ao atingir um nó terminal.

O mapeamento de caracteres para os índices do vetor (0 a 35) ocorre via função auxiliar, que retorna -1 para espaços e caracteres especiais, garantindo que sejam descartados na travessia.

3 Algoritmos e Complexidade

3.1 Normalização e Inserção na Árvore

Antes da inserção e da busca, os títulos são normalizados convertendo todos os caracteres para minúsculas e removendo espaços e caracteres não alfanuméricos. Dessa forma, buscas

como "grand theft", "GrandTheft" e "GRAND THEFT" produzem exatamente o mesmo resultado.

Após esse tratamento, o algoritmo itera sobre cada caractere. Se o ponteiro correspondente for nulo, aloca um novo `TrieNode`. O nó final da iteração recebe a flag `isEndOfTitle` verdadeira e armazena a referência do jogo.

Complexidade de Tempo: $O(L)$, onde L é o tamanho do título. O custo é estritamente proporcional à palavra inserida.

3.2 Busca e Autocomplete

A função de *autocomplete* é dividida em três fases distintas:

1. **Descida pelo Prefixo:** O algoritmo caminha na árvore seguindo o prefixo formatado. Se atingir um ponteiro nulo, a busca encerra por ausência de resultados. Tempo: $O(P)$, onde P é o tamanho do prefixo.
2. **Busca em Profundidade (DFS):** A partir do nó final do prefixo, uma varredura recursiva explora a subárvore. Cada jogo válido encontrado é armazenado em um vetor. Tempo: $O(V)$, sendo V o número de nós da subárvore explorada.
3. **Ordenação e Corte:** O vetor contendo os M jogos resgatados é ordenado e truncado para conter estritamente os k elementos requisitados.

A complexidade total de uma consulta no sistema é expressa por $O(P + V + M^2)$, em que P é o tamanho do prefixo, V representa a quantidade de nós visitados na DFS e M corresponde ao total de jogos encontrados (submetidos ao custo quadrático do método de ordenação).

3.3 Algoritmo de Ordenação (Bubble Sort)

Embora algoritmos como *Merge Sort* ou *Quick Sort* possuam melhor complexidade assintótica, optou-se por um *Bubble Sort* otimizado com a flag `trocou` devido à restrição de não utilizar bibliotecas de ordenação e ao tamanho reduzido do conjunto de resultados (M) retornados para cada consulta. Ele prioriza a popularidade e desempata pela ordem alfabética. A complexidade se mantém em $O(M^2)$ no pior caso e, ao final, o vetor passa por um `resize` para limitar a saída aos k resultados pedidos.

4 Gerenciamento de Memória

O gerenciamento manual de memória recebeu bastante atenção no desenvolvimento. O destrutor do `TrieNode` funciona de forma recursiva: ele varre as 36 posições de cada nó e vai deletando os filhos não nulos de baixo para cima, limpando a *heap* e evitando *memory leaks*. É importante destacar que os objetos da classe `Game` continuam intactos na memória, já que a árvore só se responsabiliza por gerenciar seus próprios nós e ponteiros.

5 Conclusão

A Trie demonstrou grande eficiência nas buscas por prefixo, validando sua capacidade de localização rápida independentemente do tamanho da base de dados. O único gargalo de desempenho ocorre em prefixos muito curtos (ex: uma única letra), onde o alto volume de retornos eleva consideravelmente o tempo do *Bubble Sort*. O sistema atendeu integralmente aos requisitos, processando os dados e formatando a saída de forma rápida, robusta e sem vazamentos de memória.